Table des matières

Cours 01 — Architecture de l'AI Data Cloud 1
Objectifs 1
Théorie 1
QCM — style examen (Cours 01) 11
Points clés à retenir 15

Cours 01 — Architecture de l'AI Data Cloud

Module 1/8 — Formation Snowflake (préparation SnowPro Core, COF-C03)

Comment lire ce module — C'est le plus gros morceau de l'examen : le domaine 1 pèse 31 %, et on le couvre en deux cours. Celui-ci prend l'architecture et les objets ; le Cours 02 prendra le volet IA / développement (Snowpark, Cortex…). On répond ici à *pourquoi Snowflake est bâti comme il l'est*, *quels objets existent*, *comment marche un entrepôt virtuel*, et *comment les données sont rangées et lues*. Le laboratoire vous fait voir tout ça de vos yeux.

Objectifs

À la fin de ce module, vous saurez :

- expliquer les trois couches de Snowflake et pourquoi cette architecture bat les bases « classiques » : les notions de disque partagé, sans partage et MPP ;
- situer les quatre éditions et ce qu'elles débloquent ;
- démêler les interfaces et outils : Snowsight, la CLI `snow`, SnowSQL, pilotes/connecteurs ;
- lire la hiérarchie des objets (organisation → compte → base → schéma → objets) ;
- dimensionner un entrepôt virtuel et distinguer grossir (une grosse requête) de multiplier (la concurrence) ;
- expliquer les micro-partitions, le pruning et le clustering ;
- ne plus confondre types de données et types de tables, et connaître les types de vues ;
- fixer le contexte d'une session et comprendre la précédence des paramètres.

Théorie

© 2026 iAlexMG. Tous droits réservés.

1. Les trois couches — et pourquoi elles battent une base classique

Labo : `Labo_01_SNOW.md`, étapes 2 et 3.

On l'a esquissé au Cours 00 : Snowflake sépare stockage, calcul et services cloud. Pour comprendre *pourquoi c'est malin*, il faut regarder les deux vieilles façons de faire tourner une base sur plusieurs machines.

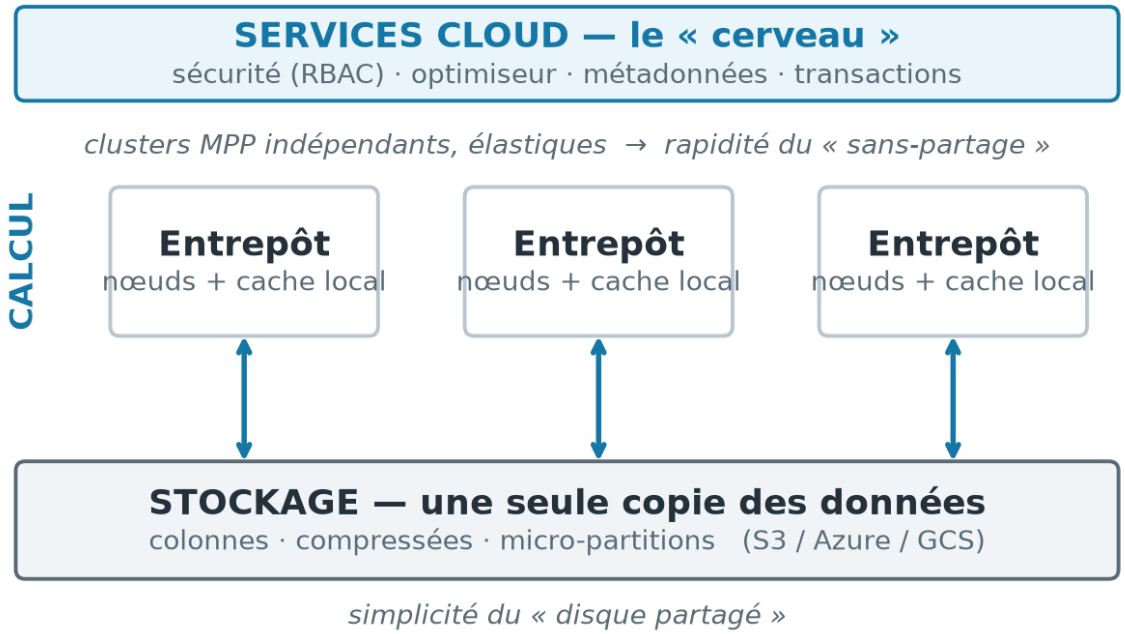
- Disque partagé (*shared-disk*) : toutes les machines lisent un seul stockage commun. Simple (une copie des données), mais ce stockage devient un goulot d'étranglement et faire écrire tout le monde au même endroit exige des verrous. La montée en charge plafonne.
- Sans partage (*shared-nothing*) : on découpe les données, un morceau par machine. Chaque nœud possède sa tranche de données + son propre CPU. Excellent pour calculer en parallèle, mais les données sont collées au calcul : agrandir oblige à tout redistribuer, et stockage et calcul grossissent ensemble.

Deux mots de vocabulaire qui reviennent à l'examen :

- Nœud = une machine ; cluster = un groupe de nœuds qui travaillent comme un tout.
- MPP (*Massively Parallel Processing*, traitement massivement parallèle) = découper une requête en morceaux exécutés en même temps sur tous les nœuds, chacun sur sa tranche, puis recoller.

Le coup de génie de Snowflake : garder le meilleur des deux mondes, le *stockage unique* du disque partagé et le *calcul parallèle* du sans-partage, mais en les découplant, de sorte qu'aucun des deux ne bride l'autre :

© 2026 iAlexMG. Tous droits réservés.



Les trois couches découplées : le calcul (des clusters MPP indépendants, avec cache local) au-dessus d'un stockage unique, les deux coiffés par les services cloud.

Ce qu'on emprunte à chacune des deux vieilles écoles :

- au disque partagé → *une seule copie des données* : simple à gérer, aucune redistribution ;
- au sans-partage → *le calcul MPP en parallèle, avec cache local* : rapide et isolé.

Stockage et calcul étant découplés, ils évoluent (et se facturent) séparément : un entrepôt peut grossir, se dédoubler ou s'endormir sans jamais toucher aux données. C'est la grande idée ; presque tout le reste en découle.

OLAP et OLTP — Les deux grandes familles de bases de données. L'OLTP (*Online Transaction Processing*, le traitement transactionnel : PostgreSQL, MySQL, Oracle…) encaisse une foule de petites écritures et lectures en lignes, à très faible latence : c'est le moteur d'une *application*. L'OLAP (*Online Analytical Processing*, le traitement analytique) agrège de gros volumes en colonnes, pour la BI et la data science. Snowflake est né OLAP (colonnes, MPP et micro-partitions, taillé pour l'analyse) et déborde vers l'OLTP avec les tables hybrides (Unistore), qu'on croise à la section 7.

Rappel des rôles de chaque couche :

© 2026 iAlexMG. Tous droits réservés.

- Stockage — vos données, une fois, en colonnes, compressées, en micro-partitions. Elles reposent sur le stockage objet du fournisseur cloud, ces services qui rangent des fichiers entiers désignés par une clé, sans arborescence de dossiers : Amazon S3, Azure Blob Storage ou Google Cloud Storage (GCS). Vous ne gérez pas de fichiers.
- Calcul — les entrepôts virtuels (*warehouses*), section 5.
- Services cloud — le « cerveau » : authentification, contrôle d'accès par rôles, optimiseur de requêtes, métadonnées, transactions. Entièrement géré, et c'est lui qui rend possibles des choses comme le cache de résultats ou le clonage sans copie.

2. Les éditions

Un compte a une édition, qui fixe les fonctions disponibles (et le coût du crédit). De la plus simple à la plus riche :

Édition	Ce qu'elle ajoute (exemples classiques à l'examen)
Standard	l'offre de base. Time Travel : 1 jour maximum.
Enterprise	Time Travel jusqu'à 90 jours, entrepôts multi-cluster, vues matérialisées, *search optimization*, masquage dynamique et sécurité par ligne.
Business Critical	sécurité et conformité renforcées : Tri-Secret Secure, PrivateLink, les conformités HIPAA et PCI DSS, la réplication pour la bascule.
Virtual Private Snowflake (VPS)	environnement isolé et dédié, pour les exigences les plus strictes.

Le Time Travel, qui sert de repère dans ce tableau, est le voyage dans le temps de Snowflake : pendant une fenêtre réglable, vous relisez une table telle qu'elle était à une heure passée, et vous ressuscitez ce qui a été supprimé par erreur. Le Fail-safe, qu'on croisera à la section 7, est le filet qui prend le relais ensuite, sept jours pendant lesquels seul le support de Snowflake peut récupérer la donnée. Les deux sont le sujet du Cours 06, section 4 ; ici, ils ne servent qu'à situer les éditions.

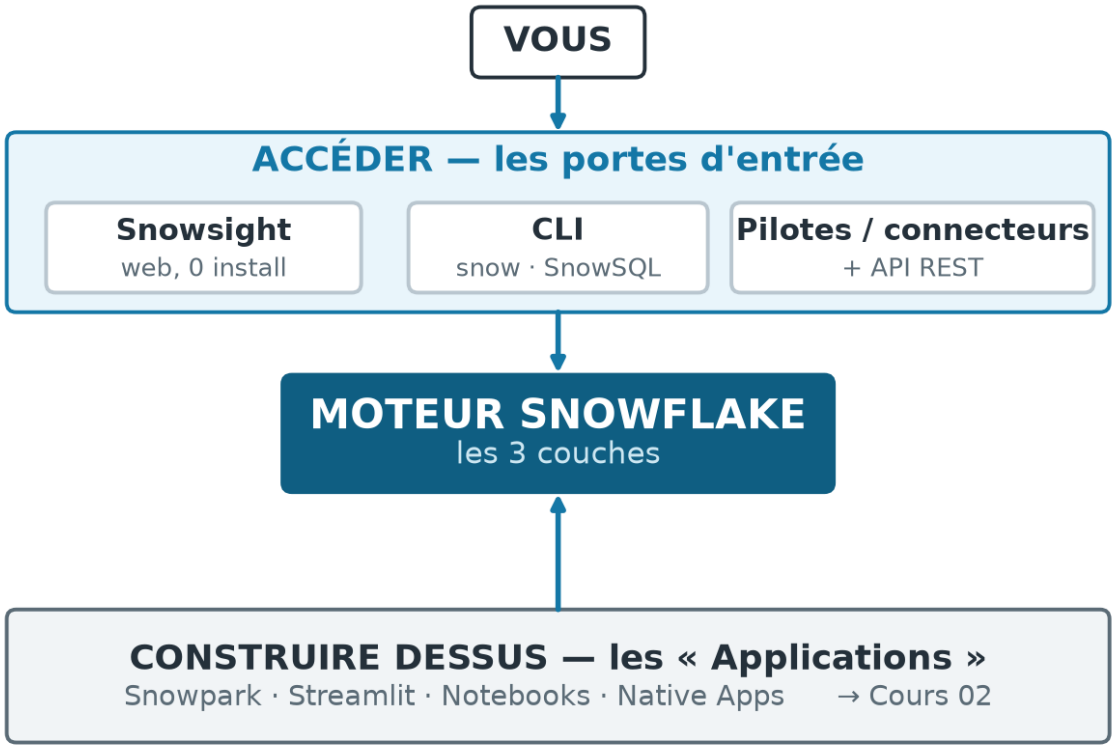
Quatre noms de la ligne Business Critical méritent leur définition. Tri-Secret Secure fait entrer une clé gérée par le client dans le chiffrement, à côté de celle de Snowflake (Cours 03, section 6). PrivateLink fait passer le trafic par un lien réseau privé du fournisseur cloud, sans sortir sur l'internet public ; le nom est celui d'AWS, Azure et Google offrent l'équivalent sous le leur. HIPAA est la loi américaine qui encadre les données de santé, et PCI DSS la norme des cartes de paiement : l'édition ne les impose pas, elle permet à un client soumis à ces textes de tenir ses engagements sur Snowflake.

Retenez surtout les seuils : multi-cluster, vues matérialisées et Time Travel > 1 jour = Enterprise et plus ; Tri-Secret Secure = Business Critical et plus. L'examen adore ces frontières. La suite de la formation écrit « Enterprise+ » pour « Enterprise et les éditions au-dessus », puisque les éditions s'emboîtent : chacune porte tout ce que porte la précédente.

3. Les interfaces et les outils

C'est un point qu'on confond souvent. Séparez deux familles : accéder à Snowflake, et construire dessus.

© 2026 iAlexMG. Tous droits réservés.



Deux familles autour du moteur : accéder (Snowsight, les CLI, les pilotes) et construire dessus (les « Applications », qui sont le Cours 02).

Accéder / travailler :

- Snowsight — l'interface web (navigateur). *L'*interface graphique : worksheets SQL, exploration, tableaux de bord, administration, Marketplace. Rien à installer. *(L'ancienne « Classic Console » est retirée.)*
- Snowflake CLI (`snow`) — la CLI moderne, installée sur votre poste : gérer des objets, lancer du SQL, déployer des applis (Streamlit, Snowpark, Native Apps).
- SnowSQL (`snowsql`) — la CLI classique, centrée SQL + chargement de fichiers (`PUT`/`GET`). C'est elle qu'utilisent beaucoup de tutoriels.

© 2026 iAlexMG. Tous droits réservés.

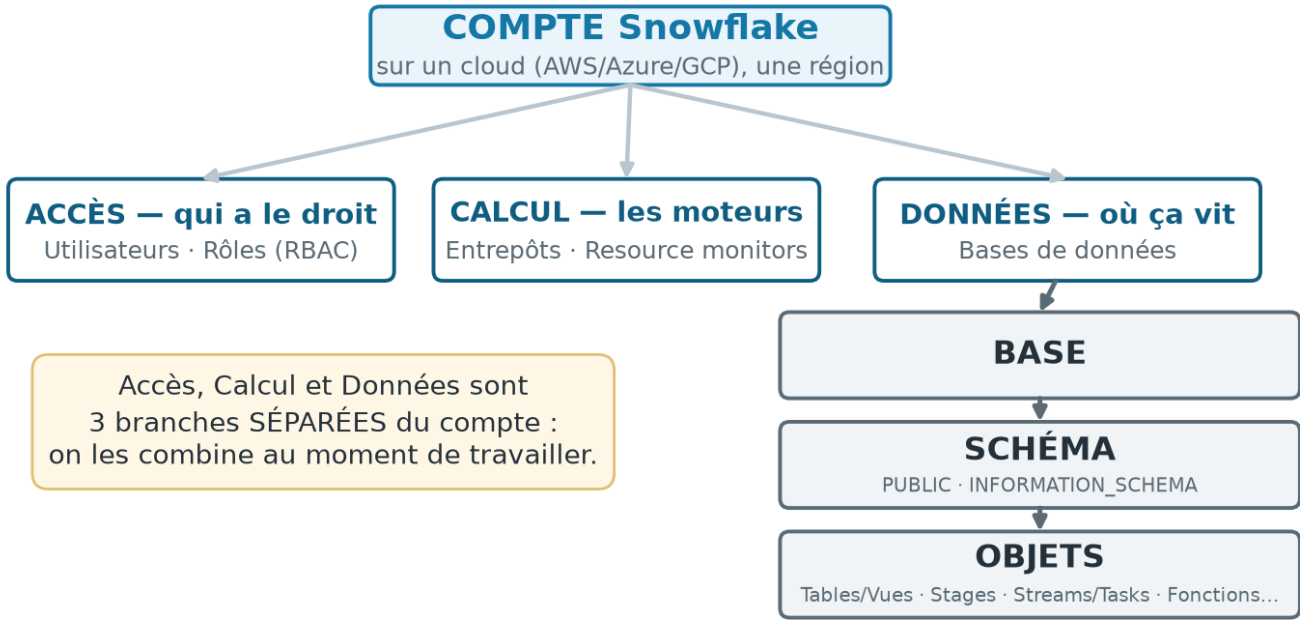
- Pilotes et connecteurs (Python, JDBC, ODBC, .NET, Go, Node, Spark, Kafka…) et API REST (dont la *SQL API*), pour qu'un programme parle à Snowflake. Un pilote est la pièce logicielle qui branche un programme sur une base de données : JDBC (*Java Database Connectivity*) est celui du monde Java, ODBC (*Open Database Connectivity*) la norme que reprennent la plupart des outils de BI sous Windows.

Construire dessus = la section « Applications » (Snowpark, Streamlit, Notebooks, Native Apps) : c'est le Cours 02.

Le raccourci : Snowsight, les CLI *ou* les pilotes sont les portes d'entrée, on passe par l'une d'elles, pas par les trois à la fois ; les Applications, c'est ce qu'on bâtit une fois entré. Et SnowSQL vs `snow` : les deux sont des CLI, mais SnowSQL = « client SQL classique », `snow` = « couteau suisse moderne orienté dev ».

4. La hiérarchie des objets

***Labo :** `Labo_01_SNOW.md`, étape 8.*



Le compte et ses trois branches séparées (accès, calcul, données), puis la descente base → schéma → objets sous la branche des données.

Deux repères utiles :

- Rôles, entrepôts et bases sont des branches séparées du compte, et on les combine au moment de travailler (les « 3 questions » de la section 8). Les données, elles, sont tout au fond : base → schéma → objet.

© 2026 iAlexMG. Tous droits réservés.

- Deux schémas spéciaux : `INFORMATION_SCHEMA` (un par base, métadonnées en lecture) et la base système `SNOWFLAKE` (dont `ACCOUNT_USAGE`, qu'on verra au Cours 05).

On s'y perd à l'écran, pour une raison simple : Snowsight ne vous affiche pas cet arbre d'un seul bloc, l'interface est rangée par tâche (travailler avec les données d'un côté, administrer de l'autre), pas selon l'arbre des objets. D'où l'impression de chercher. La correspondance :

Objet	**Où le trouver dans Snowsight**
Bases, schémas, tables, vues	Catalog › Explorer — vous dépliez base → schéma → objets
Entrepôts (*warehouses*)	Compute › Warehouses
Utilisateurs et rôles	Governance & security › Users & roles
Resource monitors (budgets de crédits)	Admin › Cost management
Masquage, tags, politiques (gouvernance)	Governance & security
Partages de données	Data sharing
Historique des requêtes	Monitoring › Query History

À retenir : les données vivent sous Catalog ; les moteurs (entrepôts) sous Compute ; qui a le droit (utilisateurs et rôles) sous Governance & security. Ce sont exactement les trois branches séparées du compte, l'interface les range simplement dans trois coins différents. *(Les libellés de menus bougent : Snowflake rafraîchit Snowsight régulièrement, et ce tableau suit sa documentation, relevée le 2026-09-20.)*

Ce que vous voyez *vraiment* en ouvrant l'explorateur, et pourquoi ça semble touffu. La liste porte plusieurs bases que vous n'avez pas créées : ce sont des bases système et des exemples livrés par Snowflake. Décodage de l'écran :

Ce que vous voyez	**C'est quoi**
SNOWFLAKE	base système, lecture seule : métadonnées et usage du compte (`ACCOUNT_USAGE`…), Cours 05
SNOWFLAKE_SAMPLE_DATA	jeux d'exemple partagés, dont les schémas `TPCH_SF1` … `SF1000` et `TPCDS`… (données de démo, de plus en plus grosses)
une base au nom en `USER$`…	la base personnelle que Snowflake attache à votre utilisateur ; elle sert à la plateforme, et n'accueille pas vos tables
une base par tutoriel suivi	un quickstart laisse derrière lui sa base de démonstration, sous le nom qu'il a choisi

© 2026 iAlexMG. Tous droits réservés.

Ce que vous voyez	C'est quoi
vos propres bases	celles que vous créez, dont `SNOWPRO_DB` au labo de cette formation

Et dans chaque base reviennent les deux mêmes schémas de service : `PUBLIC` (là où atterrissent vos objets par défaut) et `INFORMATION_SCHEMA` (le catalogue de la base, en lecture). Ce dernier explique un compteur qui surprend : une base neuve affiche déjà plusieurs dizaines de vues, qui sont *ses* vues de métadonnées, pas les vôtres.

Donc rien d'anormal : la plupart de ces bases ne sont pas à vous. Pour vous y retrouver, dépliez la vôtre (au labo : `SNOWPRO_DB` → schéma `SANDBOX` → vos tables). Le reste, c'est le décor système.

Comment *voir* le contenu d'une base, et pourquoi la vôtre semble « vide ». Dépliez-la dans l'explorateur (base → schéma → objets), ou en SQL : `SHOW SCHEMAS IN DATABASE <base>;`, puis `SHOW TABLES IN SCHEMA <base>.<schéma>;`, et `SELECT * FROM <base>.<schéma>.<table>;`. Si un schéma affiche « No objects found », ce n'est pas un bug : il est vide tant que vous n'y créez rien. Une base ne « contient » rien d'elle-même : c'est vous qui y déposez des tables (voir la section 8). *(Ne confondez pas non plus une base avec un* worksheet *: le `.sql` est juste l'éditeur où vous tapez vos requêtes ; la base est l'endroit où atterrissent les objets.)*

5. Les entrepôts virtuels en détail

Labo : `Labo_01_SNOW.md`, étape 2.

Un entrepôt virtuel est un cluster MPP qui exécute requêtes et chargements. C'est le seul poste de coût « à l'usage », alors autant le maîtriser.

- Tailles. `XS`, `S`, `M`, `L`, `XL`, `2XL`… jusqu'à `6XL`. Chaque cran double le nombre de nœuds, et le coût : à génération égale, `S` coûte le double d'un `XS`, `M` le quadruple, `L` le huituple. C'est ce rapport que l'examen teste, pas le prix. Pour la formation, `XS` suffit largement.
- Facturation à la seconde (minimum 60 s au réveil), seulement quand l'entrepôt tourne. `AUTO_SUSPEND` l'endort après N secondes d'inactivité ; `AUTO_RESUME` le réveille.
- Grossir contre multiplier — la distinction reine :
  - Grossir (*scale up*, taille supérieure) accélère une requête lourde : plus de nœuds pour le même travail.
  - Multiplier (*scale out*, entrepôt multi-cluster) sert la concurrence : plusieurs clusters pour absorber beaucoup d'utilisateurs en même temps. Requiert Enterprise+, avec une *scaling policy* (`Standard` réactive, `Economy` économe).
- Générations. Un `CREATE WAREHOUSE` standard qui ne dit rien crée aujourd'hui un entrepôt de deuxième génération (Gen2) : la documentation pose `GENERATION = '2'` par défaut, sauf dans une région qui ne l'offre pas. Gen2 abat plus de travail à taille égale, et son heure coûte plus de crédits que celle de la génération précédente ; il n'existe pas en `5XL` ni en `6XL`. Le tarif de chaque taille et de chaque génération vit dans la table de consommation de Snowflake, qui fait foi, et qui varie selon le cloud et la région : un `XS` Gen2 y était à 1,35 crédit l'heure sur AWS le 2026-09-18. C'est donc l'entrepôt que le Labo 00 vous a fait créer.

© 2026 iAlexMG. Tous droits réservés.

- Variantes. Les entrepôts Snowpark-optimized donnent plus de mémoire par nœud, pour le ML et les charges gourmandes.

Réflexe de production : un entrepôt par usage — On sépare souvent le calcul du chargement, de la transformation et de la BI, chacun sur son entrepôt, pour dimensionner et surveiller les coûts séparément.

6. Le stockage : micro-partitions, pruning, clustering

Labo : `Labo_01_SNOW.md`, étape 4.

Vous ne réglez presque rien (Snowflake fait le travail), mais l'examen teste ces mécanismes :

- Micro-partitions. Snowflake découpe automatiquement chaque table en petits blocs (~50 à 500 Mo de données non compressées), stockés en colonnes et compressés. Pour chaque bloc, il retient des métadonnées (min/max, nombre de valeurs distinctes…).
- Pruning (élagage). Grâce à ces bornes, une requête filtrée saute les blocs qui ne peuvent pas contenir de résultat. Gratuit, automatique, invisible, et c'est le grand levier de performance. Le *Query Profile* (Cours 05) montre « partitions scannées / total ».
- Clustering. Les données sont naturellement rangées dans l'ordre de chargement. Sur de très grosses tables, on peut définir une clé de clustering pour regrouper les lignes souvent filtrées ensemble → meilleur pruning. Le service Automatic Clustering l'entretient. Levier avancé, inutile à petite échelle.

7. Types de données ≠ types de tables (+ types de vues)

Labo : `Labo_01_SNOW.md`, étapes 5, 6 et 7.

Piège fréquent : ce sont deux axes indépendants, pas une correspondance.

Types de données = la *nature du contenu* :

- structuré — lignes/colonnes classiques (`NUMBER`, `STRING`, `DATE`…) ;
- semi-structuré — JSON, Avro, Parquet, XML → dans une colonne `VARIANT` (ou `OBJECT`/`ARRAY`) ;
- non structuré — fichiers (PDF, images, audio) → gardés dans des stages, listés par une directory table et lus par une URL de fichier (Cours 04, section 8).

Types de tables = deux axes, encore une fois. La page officielle *key concepts* range surtout les tables par nature / stockage ; l'examen, lui, teste aussi leur persistance (durée de vie et filets de sécurité). Ne confondez pas les deux.

Axe A — la persistance (Time Travel + Fail-safe) :

Type	Time Travel	Fail-safe	Pour quoi
Permanente (défaut)	0–90 j (édition)	7 j	données de référence

© 2026 iAlexMG. Tous droits réservés.

Type	Time Travel	Fail-safe	Pour quoi
Transient	0–1 j	non	données rejouables (moins cher)
Temporaire	0–1 j (dans la session)	non	jetable, le temps d'une session

Axe B — la nature / le stockage (la classification de la doc) :

Type	En un mot
Snowflake (standard)	le cas normal : stockage interne, colonnes, micro-partitions
Externe (*external*)	lit des fichiers externes (S3/Azure/GCS) en lecture seule
Iceberg	format ouvert Apache Iceberg, sur un stockage cloud que vous gérez
Hybride (*Unistore*)	en lignes, faible latence, clés/contraintes → charges transactionnelles (OLTP)
Dynamique	se rafraîchit toute seule depuis une requête (outil de transformation)

La clé : les deux axes se croisent. Une table est *à la fois* d'une certaine persistance (axe A) et d'un certain stockage (axe B), le cas courant étant permanente + Snowflake standard. Et une même table permanente peut mêler des colonnes structurées ET une colonne `VARIANT`. « Permanente / transient / temporaire » dit combien de temps elle vit ; « standard / externe / Iceberg / hybride / dynamique » dit où et comment elle est rangée. Deux questions distinctes.

Types de vues (une vue = une requête enregistrée) :

- standard — ne stocke rien, recalculée à chaque lecture ;
- matérialisée — stocke le résultat précalculé, entretenu automatiquement (Enterprise+, avec des restrictions) ;
- sécurisée (*secure*) — cache sa définition et désactive certaines optimisations qui pourraient fuiter des données ; utile pour le partage.

8. Session et contexte

Labo : `Labo_01_SNOW.md`, étape 1.

Avant toute exécution, Snowflake veut savoir quel rôle, quel entrepôt, quelle base et quel schéma : les 3 questions de l'aide-mémoire, qu'on fixe avec `USE` :

```
USE ROLE SYSADMIN;
USE WAREHOUSE SNOWPRO_WH;
USE DATABASE SNOWPRO_DB;
USE SCHEMA SANDBOX;
```


© 2026 iAlexMG. Tous droits réservés.

Pourquoi fixer le contexte AVANT de créer. Tout objet que vous créez ensuite atterrit dans la base et le schéma courants. Avec le contexte ci-dessus, `CREATE TABLE MENU (...)` crée exactement `SNOWPRO_DB.SANDBOX.MENU` : c'est ça, « la table correspond à la base », rien de magique, c'est le contexte qui la range. Changez le `USE SCHEMA`, et le même `CREATE` atterrit ailleurs.

Créer une table n'est pas la remplir — Trois choses à séparer, c'est *la* source de confusion au début, et une table n'est pas non plus un fichier : 1. La structure (les colonnes) — c'est vous qui la définissez dans `CREATE TABLE (...)`. Les noms de colonnes ne sortent pas tout seuls d'un fichier de données. Dans un *quickstart*, ils vous sont fournis (l'auteur connaît le format du fichier source, ex. sur S3). Sinon : vous inspectez l'en-tête du fichier, ou vous laissez Snowflake inférer le schéma (`INFER_SCHEMA`). 2. L'emplacement — la base et le schéma, décidés par le contexte ci-dessus. 3. Le contenu (les données) — il arrive après, par un chargement (`COPY INTO` depuis un *stage*) : c'est tout le Cours 04. En clair : `CREATE TABLE` fabrique la coquille vide ; `COPY INTO` la remplit et fait correspondre colonnes ↔ fichier.

Beaucoup de comportements se règlent par des paramètres, à trois niveaux avec une précédence (le plus proche gagne) :

```
COMPTE   <   UTILISATEUR   <   SESSION      (SESSION l'emporte)
```

Autrement dit, un paramètre fixé pour la session écrase la valeur de l'utilisateur, qui écrase celle du compte. (Il existe aussi des *paramètres d'objet*, portés par l'objet lui-même.) `SHOW PARAMETERS` affiche les valeurs en vigueur, on le voit au labo.

QCM — style examen (Cours 01)

Comment répondre — Faites les neuf questions avant d'ouvrir le corrigé, qui commence à la page suivante. Une question à choix multiple dit combien de réponses cocher, comme à l'examen. Comptez une minute par question.

1. L'équipe de la BI se plaint : ses tableaux de bord ralentissent chaque fois que l'équipe d'ingénierie lance ses chargements. Les deux travaillent sur les mêmes tables. Que permet l'architecture de Snowflake ?
    a. Copier les tables dans une seconde base, pour que chaque équipe lise la sienne.
    b. Donner à chaque équipe son entrepôt, les deux lisant la même et unique copie des données.
    c. Répartir les micro-partitions de chaque table entre deux clusters, un par équipe.
    d. Basculer les tables en `transient`, moins coûteuses à lire à plusieurs.
2. Un architecte habitué aux bases sans partage vous demande comment Snowflake ajoute de la puissance sans redistribuer les données entre les nœuds. Que répondez-vous ?
    a. Il les redistribue quand même, en tâche de fond, aux heures creuses.
    b. Chaque nœud garde une copie complète de la table, ce qui rend la redistribution inutile.
    c. La redistribution est faite par la couche des services cloud, à chaque requête.

© 2026 iAlexMG. Tous droits réservés.

d. Les données ne sont pas collées au calcul : elles vivent une fois dans un stockage commun, que des clusters MPP indépendants lisent.

3. Votre compte est en édition Standard. On vous demande un Time Travel de 30 jours sur une table de référence, et un entrepôt multi-cluster pour absorber les rapports du lundi matin.

   a. Ni l'un ni l'autre : les deux demandent l'édition Enterprise au minimum.

   b. Les deux : il suffit de les activer, l'édition ne joue que sur le prix du crédit.

   c. Le Time Travel de 30 jours oui, le multi-cluster non.

   d. Le multi-cluster oui, le Time Travel de 30 jours non.

4. *(choix multiple, sélectionnez-en DEUX)* Une agrégation sur une très grosse table met huit minutes sur un entrepôt `SMALL`, alors qu'aucune autre requête ne tourne. Le profil montre que presque toutes les micro-partitions ont été lues. Deux gestes peuvent aider.

   a. Grossir l'entrepôt d'un ou deux crans.

   b. Passer l'entrepôt en multi-cluster.

   c. Ranger la table par une clé de clustering sur la colonne filtrée.

   d. Basculer la table en `transient`, pour qu'elle porte moins d'historique à lire.

5. Vous devez déposer un fichier de votre poste dans un stage interne. Votre collègue, lui, veut seulement écrire du SQL depuis son navigateur, sans rien installer.

   a. Vous passez tous les deux par Snowsight, qui sait aussi téléverser un fichier local par `PUT`.

   b. Vous passez tous les deux par une CLI, Snowsight ne servant qu'à l'administration du compte.

   c. Votre collègue reste dans Snowsight ; vous, il vous faut SnowSQL, la CLI `snow` ou un pilote, car `PUT` ne s'exécute pas depuis un *worksheet*.

   d. Vous passez par la SQL API en REST, seule voie pour envoyer un fichier local.

6. Vous créez une table, la commande réussit, et vous ne la retrouvez pas dans la base que vous dépliez. Où est-elle ?

   a. Dans la base système `SNOWFLAKE`, où atterrissent les objets créés hors contexte.

   b. Nulle part : un `CREATE TABLE` sans base explicite échoue sans message.

   c. Dans `INFORMATION_SCHEMA`, le schéma qui reçoit les objets par défaut.

   d. Dans la base et le schéma courants au moment du `CREATE`, ceux qu'ont fixés les `USE`.

© 2026 iAlexMG. Tous droits réservés.

7. Une table de travail est reconstruite chaque nuit depuis sa source, et se rejoue en une heure si on la perd. Quel type de table retenir, et qu'est-ce qu'on y perd ?
    a. `transient` : pas de Fail-safe, donc moins chère au stockage, et un Time Travel qui plafonne à un jour.
    b. Temporaire : elle survivra aux sessions tant que la tâche de nuit la réécrit.
    c. Permanente : c'est le seul type qu'une tâche planifiée sait écrire.
    d. Externe : une table reconstruite chaque nuit doit pointer vers les fichiers de sa source.
8. Vous partagez des données à un autre compte, mais la requête qui les prépare porte votre logique métier et doit rester invisible. Que partagez-vous ?
    a. Une vue standard : sa définition n'est de toute façon pas lisible par un consommateur.
    b. Une vue matérialisée : le résultat étant stocké, la définition ne sert plus.
    c. Une vue sécurisée : elle cache sa définition et renonce aux optimisations qui pourraient laisser deviner les données.
    d. Un clone de la table, dont vous supprimez l'historique avant de le partager.
9. Le paramètre `USE_CACHED_RESULT` vaut `TRUE` au niveau du compte. Vous le passez à `FALSE` pour votre session, le temps d'un test de performance. Quelle valeur s'applique, et à qui ?
    a. Celle du compte, `TRUE` : une session ne contredit pas un paramètre de compte.
    b. Celle de la session, `FALSE`, pour vos requêtes seulement et jusqu'à la fin de la session.
    c. Celle de la session, `FALSE`, mais pour tout le compte : un paramètre est global.
    d. Aucune : Snowflake lève une erreur tant que les deux niveaux ne concordent pas.

© 2026 iAlexMG. Tous droits réservés.

Corrigé expliqué — Cours 01

À lire après avoir répondu. Chaque réponse nomme la section à relire : ce que vous ratez ici vous dit quoi reprendre.

1. Réponse b. Le calcul étant découplé du stockage, plusieurs entrepôts lisent la même copie des données en même temps, sans se gêner : c'est le réflexe de production dit « un entrepôt par usage ». L'option a résoudrait le conflit en dupliquant les données, ce que justement on n'a pas à faire. L'option c décrit un partitionnement à la main qui n'existe pas ici. L'option d change la persistance de la table, ce qui ne touche ni la concurrence ni la vitesse (sections 1 et 5).

2. Réponse d. Le stockage unique vient du disque partagé, le calcul MPP en parallèle vient du sans-partage, et le découplage fait qu'ajouter du calcul ne déplace aucune donnée. Les options a et c décrivent une redistribution qui n'a pas lieu, et l'option b décrit une réplication complète par nœud, qui n'est ni l'une ni l'autre des deux écoles (section 1).

3. Réponse a. Time Travel au-delà d'un jour et entrepôts multi-cluster sont deux fonctions d'Enterprise et plus, au même titre que les vues matérialisées et le *search optimization*. En Standard, le Time Travel plafonne à un jour et l'entrepôt reste à un seul cluster. L'édition ne fait donc pas que changer le prix du crédit, contrairement à l'option b (section 2).

4. Réponses a et c. Une requête seule sur le compte ne fait pas la queue : le mal est la quantité de données lues et la mémoire disponible, donc on grossit l'entrepôt et on range la table pour qu'elle s'élague. Le multi-cluster de l'option b sert la concurrence, et il n'y en a pas ici. L'option d confond la persistance avec le volume lu : une table `transient` porte les mêmes micro-partitions (sections 5 et 6).

5. Réponse c. `PUT` dépose un fichier local dans un stage interne, et il ne s'exécute pas depuis un *worksheet* : il faut un client installé, SnowSQL, la CLI `snow` ou un pilote. Votre collègue, lui, n'a besoin que du navigateur. L'option a prête à Snowsight une capacité qu'il n'a pas pour `PUT`, l'option b lui retire des capacités qu'il a, et la SQL API de l'option d soumet du SQL, pas des fichiers (section 3).

6. Réponse d. Tout objet créé atterrit dans la base et le schéma courants : c'est le contexte, fixé par `USE DATABASE` et `USE SCHEMA`, qui range la table, et rien d'autre. Les options a et c désignent des schémas système en lecture, qui ne reçoivent pas vos objets ; l'option b prête à Snowflake un échec silencieux, alors qu'il refuse la commande quand aucun contexte ne tient (section 8).

7. Réponse a. Une table rejouable est le cas d'usage de `transient` : on renonce aux sept jours de Fail-safe, donc à une part du coût de stockage, et le Time Travel y plafonne à un jour. La table temporaire de l'option b meurt avec la session qui l'a créée. L'option c invente une restriction sur les tâches. L'option d confond persistance et stockage : une table externe lit des fichiers en place, elle ne se reconstruit pas (section 7).

8. Réponse c. La vue sécurisée est faite pour ça : elle cache sa définition et désactive des optimisations qui pourraient laisser fuiter les lignes sous-jacentes, ce qui en fait la forme normale du partage. Une vue standard expose sa définition à qui peut la décrire, ce qui écarte l'option a. L'option b confond stockage du résultat et secret de la définition. L'option d copie les données au lieu de donner accès (section 7).

9. Réponse b. La précédence va du plus lointain au plus proche : compte, puis utilisateur, puis session, et le plus proche l'emporte. Votre `ALTER SESSION` ne vaut que pour vos requêtes et tombe à la fin de la session, ce qui écarte les options a et c. Aucun conflit n'est levé : la précédence existe justement pour trancher sans erreur, contre l'option d (section 8).

© 2026 iAlexMG. Tous droits réservés.

Points clés à retenir

- Snowflake = trois couches découplées (stockage / calcul / services cloud) ; il marie disque partagé (une copie des données) et sans partage (clusters MPP indépendants). Stockage et calcul se facturent séparément.
- Quatre éditions : Standard → Enterprise → Business Critical → VPS. Seuils classiques : multi-cluster / vues matérialisées / Time Travel > 1 j = Enterprise+ ; Tri-Secret Secure = Business Critical+.
- Interfaces : *accéder* (Snowsight web, CLI `snow`, SnowSQL, pilotes/REST) vs *construire* (Snowpark, Streamlit, Notebooks, Native Apps → Cours 02).
- Hiérarchie : compte → (rôles / entrepôts / bases, branches séparées) → base → schéma → objets.
- Entrepôt : payé à la seconde quand il tourne ; chaque taille double les crédits ; grossir = une grosse requête, multiplier (multi-cluster) = la concurrence.
- Stockage : micro-partitions automatiques + pruning (bornes min/max) = la performance ; le clustering est un levier avancé.
- Types de données (structuré / semi / non structuré) ≠ types de tables (permanente / transient / temporaire / externe / dynamique / Iceberg) : deux axes indépendants. Vues : standard / matérialisée / sécurisée.
- Contexte = rôle + entrepôt + base/schéma (`USE`) ; précédence des paramètres : compte < utilisateur < session.

© 2026 iAlexMG. Tous droits réservés.